

MediPilot Implementation README

Architecture, Tech Stack, and Execution

Team 01 BIT

August 29, 2026

Abstract

This document serves as the implementation README, outlining the solution architecture, technology stack, directory structure, synthetic data production, and the specific role of the Large Language Model (LLM). It incorporates the system's architecture diagrams directly to provide a comprehensive, visual guide to the system.

Contents

1 Solution Architecture

The system enforces a strict boundary between the patient/nurse interfaces and the clinical risk engine.

1.1 Frontend (Web Layer)

The frontend (located in `web/`) is a Next.js 16 application that houses all seven user surfaces, including the interactive kiosk and the nurse board. It manages the talking, branching triage conversation, speech-to-text integration, and local offline phrase matching without performing any clinical evaluation.

1.2 Backend (Triage Engine)

The backend (located in `backend/`) is a Python/FastAPI orchestrator. It contains the deterministic Band Engine, the gradient-boosted Risk Model, and the strict safety floors (Red Flags, SpO2 Bias Guards).

2 Tech Stack

The system runs on commodity hardware to comply with the DPDP Act 2023, specifically avoiding mandatory paid cloud GPU dependencies.

- **Frontend:** Next.js 16, React 19, Tailwind CSS 4, `webkitSpeechRecognition`.
- **Backend:** Python 3.10+, FastAPI, Uvicorn, Scikit-learn, Pandas.
- **Machine Learning:** `faster-whisper` (CTranslate2, int8), Llama.cpp / Ollama (for offline structural parsing).

3 Role of the LLM

The Large Language Model (LLM) is strictly confined to **Track A (Perception Layer)**.

- **What it does:** The LLM translates unstructured patient narrative into a structured JSON schema (e.g., extracting symptoms and pain levels).
- **What it NEVER does:** The LLM structurally cannot produce an acuity score, assign a triage band, or make clinical judgements.

This extraction falls back to a deterministic, keyword-based structurizer if inference resources are constrained.

4 Production of Synthetic Data

All data within the repository is entirely synthetic. The 20-record demonstration corpus (located at `web/lib/seed/corpus.ts` and `backend/data/corpus_20.json`) was generated deterministically to test specific edge cases, such as wait-ceiling breaches, out-of-distribution (OOD) gates, and zero-history ABHA paths. No actual Patient Health Information (PHI) was used.

5 Directory Structure

The repository is structured to enforce separation of concerns:

```
.
|-- web/                # Next.js frontend (patient kiosk, board)
|-- backend/           # Python FastAPI engine (risk model, invariants)
|-- docs/              # Documentation and reports
|   |-- README.md      # Implementation markdown
|   |-- paper/          # LaTeX whitepaper source
|   |-- diagrams/       # SVG and PNG architecture diagrams
|   \-- submission/     # Compiled PDFs for evaluation
|-- archive/           # Quarantined legacy code
\-- .migration/        # Hashes and logs proving migration integrity
```

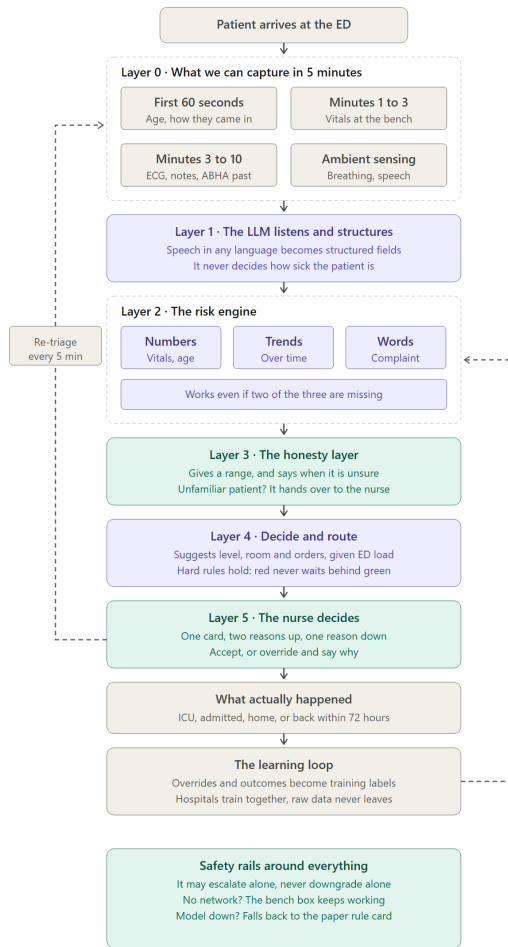


Figure 1: Full End-to-End Architecture of MediPilot

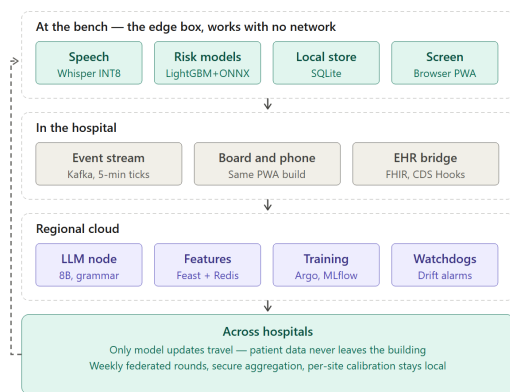


Figure 2: Tech Stack and the Three Tiers of Fallback Inference

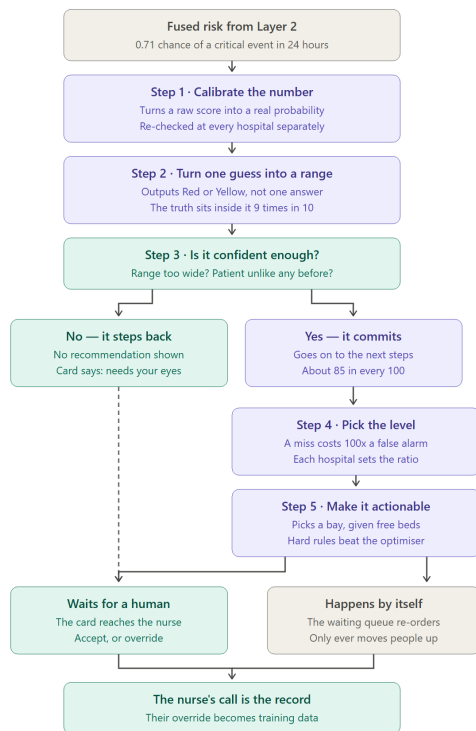


Figure 3: The Decision Path: The model reports what was said, a fixed table decides what it means.